Buổi 4

Lớp và đối tượng

Học phần: Lập trình hướng đối tượng

Nội dung bài học

1. Từ struct đến class

1.1 Cài đặt kiểu dữ liệu bằng struct và hạn chế
1.2 Định nghĩa lớp
1.3 Định nghĩa hàm thành viên và chương trình hoàn chỉnh
1.4 Biểu diễn lớp bằng sơ đồ UML

2. Phạm vi lớp và truy cập thành viên

2.1 Phạm vi lớp và các cách truy cập thành viên
2.2 Định nghĩa hàm thành viên trong và ngoài lớp

3. Điều khiển truy cập

3.1 Các nhãn public, private và protected
3.2 Lợi ích của điều khiển truy cập – ví dụ thay đổi cài đặt

4. Hàm truy cập và hàm tiện ích

4.1 Hàm truy cập
4.2 Ví dụ hoàn chỉnh: lớp PhanSo
4.3 Phân tích thiết kế lớp PhanSo
4.4 Lỗi thường gặp khi định nghĩa lớp

5. Tổng kết và thực hành

5.1 Tóm tắt buổi 4
5.2 Bài thực hành buổi 4 – Cài đặt lớp Time và PhanSo
5.3 Câu hỏi và bài tập

Ví dụ 4.1 – Kiểu Time bằng struct (theo giáo trình)

```
struct Time
{  int hour;     // 0 - 23
   int minute;   // 0 - 59
   int second;   // 0 - 59
};
void PrintMilitary(const Time &t);   // HH:MM
void PrintStandard(const Time &t);   // HH:MM:SS AM/PM
int main()
{  Time DinnerTime;
   DinnerTime.hour = 18;
   DinnerTime.minute = 30;
   DinnerTime.second = 0;
   cout << "Dinner will be held at ";
   PrintMilitary(DinnerTime);
   cout << " military time,\nwhich is ";
   PrintStandard(DinnerTime);
   cout << " standard time.\n";
   DinnerTime.hour = 29;  DinnerTime.minute = 73;
   cout << "\nTime with invalid values: ";
   PrintMilitary(DinnerTime);
}
```

- **Kết quả chạy:**

```
Dinner will be held at 18:30 military time,
which is 6:30:00 PM standard time.

Time with invalid values: 29:73
```

- **Các hạn chế của struct:**
  - Dữ liệu **không được bảo vệ**: bất kỳ đâu cũng gán được `hour = 29` ⇒ đối tượng ở trạng thái **sai** (29:73) mà không ai phát hiện.
  - **Không có khởi tạo tự động**: biến Time vừa khai báo chứa giá trị rác.
  - Dữ liệu và hàm xử lý **tách rời**: PrintMilitary đứng ngoài, nhận Time làm tham số.
  - Đổi cách biểu diễn (ví dụ lưu số giây kể từ 0 giờ) ⇒ **mọi** chương trình dùng `t.hour` phải sửa.

1.2 Định nghĩa lớp (class)

Lớp trong C++

Lớp là kiểu dữ liệu do người dùng định nghĩa, gồm các **thành viên dữ liệu** (data members – thuộc tính) và các **hàm thành viên** (member functions – phương thức), kèm các **nhãn điều khiển truy cập** `public`, `private`, `protected`.

```
class Time
{  public:                          // giao diện
      Time();                       // constructor
      void SetTime(int, int, int);
      void PrintMilitary();
      void PrintStandard();
   private:                         // cài đặt
      int Hour;    // 0 - 23
      int Minute;  // 0 - 59
      int Second;  // 0 - 59
};                                  // chú ý dấu ;
```

- **Constructor** `Time()`: hàm đặc biệt cùng tên lớp, **tự động gọi** khi tạo đối tượng để khởi tạo dữ liệu (buổi 5).

- **Cú pháp tổng quát:**

```
class TenLop
{  public:
      // các thành viên ai cũng truy cập được
   private:
      // chỉ hàm thành viên (và friend) truy cập
   protected:
      // như private + lớp dẫn xuất (buổi 9)
};
```

- **Quy ước đặt tên:** tên lớp viết hoa chữ đầu (Time, PhanSo, SinhVien); thường đặt phần `public` trước để người đọc thấy **giao diện** ngay.

- **Khai báo đối tượng:**

```
Time Sunset;                // đối tượng
Time ArrayOfTimes[5];       // mảng đối tượng
Time *PointerToTime;        // con trỏ tới đối tượng
Time &DinnerTime = Sunset; // tham chiếu tới đối tượng
```

```
Time::Time() { Hour = Minute = Second = 0; }
void Time::SetTime(int h, int m, int s)
{  Hour   = (h >= 0 && h < 24) ? h : 0;
   Minute = (m >= 0 && m < 60) ? m : 0;
   Second = (s >= 0 && s < 60) ? s : 0;
}
void Time::PrintMilitary()
{  cout << (Hour < 10 ? "0" : "") << Hour << ":"
        << (Minute < 10 ? "0" : "") << Minute;
}
void Time::PrintStandard()
{  cout << ((Hour == 0 || Hour == 12) ? 12 : Hour % 12)
        << ":" << (Minute < 10 ? "0" : "") << Minute
        << ":" << (Second < 10 ? "0" : "") << Second
        << (Hour < 12 ? " AM" : " PM");
}
int main()
{  Time t;                          // constructor được gọi
   cout << "The initial military time is ";
   t.PrintMilitary();
   ...
   t.SetTime(13, 27, 6);  ...
   t.SetTime(99, 99, 99); ...      // giá trị sai
}
```

- **Kết quả chạy:**

```
The initial military time is 00:00
The initial standard time is 12:00:00 AM

Military time after SetTime is 13:27
Standard time after SetTime is 1:27:06 PM

After attempting invalid settings:
Military time: 00:00
Standard time: 12:00:00 AM
```

- **So với struct (ví dụ 4.1):**
  - Đối tượng mới tạo có giá trị **00:00** (không rác) nhờ constructor.
  - `SetTime(99, 99, 99)` bị **kiểm tra** ⇒ đặt về 0, đối tượng **luôn hợp lệ**.
  - Gán trực tiếp `t.Hour = 29;` ⇒ **lỗi biên dịch** (Hour là private).
- `Time::` – toán tử phạm vi: cho biết SetTime là thành viên của lớp Time.

1.4 Biểu diễn lớp bằng sơ đồ UML

- **UML** (Unified Modeling Language) – ngôn ngữ mô hình hóa chuẩn; **sơ đồ lớp** gồm 3 ngăn: tên lớp – thuộc tính – phương thức. Ký hiệu truy cập: **+** public, **−** private, # protected.

```
┌──────────────────────────────────┐
│              Time                │
├──────────────────────────────────┤
│ - Hour   : int                   │
│ - Minute : int                   │
│ - Second : int                   │
├──────────────────────────────────┤
│ + Time()                         │
│ + SetTime(h, m, s : int)         │
│ + PrintMilitary()                │
│ + PrintStandard()                │
└──────────────────────────────────┘
```

- Lớp PhanSo: các hàm tiện ích UCLN, RutGon ghi dấu **−**; Set, GetTu, Cong, In ghi dấu **+**.

- **Đọc sơ đồ ⇒ viết mã:** mỗi dòng "−" là thành viên trong vùng `private:`, mỗi dòng "+" là thành viên trong vùng `public:`; kiểu đứng sau dấu `:`.

Ví dụ 4.8 – Từ sơ đồ sang C++

```
┌──────────────────────────┐
│        HinhTron          │
├──────────────────────────┤
│ - BanKinh : double       │
├──────────────────────────┤
│ + SetBanKinh(r:double)   │
│ + DienTich() : double    │
└──────────────────────────┘
```

```
class HinhTron
{  private:
      double BanKinh;
   public:
      void SetBanKinh(double r)
      {  BanKinh = (r > 0) ? r : 1;  }
      double DienTich() { return 3.14159*BanKinh*BanKinh; }
};
```

2.1 Phạm vi lớp và các cách truy cập thành viên

- **Phạm vi lớp:** các thành viên dữ liệu và hàm thành viên thuộc **phạm vi của lớp**.
  - Bên **trong** hàm thành viên: gọi thẳng tên thành viên (`Hour`, `SetTime(…)`).
  - Bên **ngoài** lớp: truy cập thành viên public qua **đối tượng**, **tham chiếu** hoặc **con trỏ**.

Truy cập qua	Toán tử	Ví dụ
Đối tượng	`.`	`t.SetTime(7, 5, 0);`
Tham chiếu	`.`	`r.PrintMilitary();`
Con trỏ	`->`	`p->SetTime(7, 5, 0);`
Con trỏ (giải tham chiếu)	`(*p).`	`(*p).PrintStandard();`

- Biến cục bộ trong hàm thành viên **trùng tên** thành viên dữ liệu sẽ **che** thành viên; truy cập thành viên bị che bằng `Time::Hour` hoặc `this->Hour` (buổi 6).

Ví dụ 4.2 – Ba cách truy cập

```
Time t;
Time *p = &t;
Time &r = t;
p->SetTime(7, 5, 0);       // qua con trỏ
r.PrintMilitary();         // qua tham chiếu
cout << " ";
(*p).PrintStandard();      // giải tham chiếu con trỏ
cout << endl << sizeof(Time);
```

```
07:05 7:05:00 AM
12
```

- `sizeof(Time)` = **12** = 3 × sizeof(int): **chỉ** thành viên dữ liệu chiếm chỗ trong đối tượng; các hàm thành viên được lưu **một bản duy nhất** dùng chung cho mọi đối tượng.

- **Cách 1 – định nghĩa bên trong khai báo lớp:** hàm tự động được xem là **inline**. Dùng cho hàm **ngắn** (get/set).

```
class Time
{  public:
      int GetHour() { return Hour; }
      ...
};
```

- **Cách 2 – khai báo prototype trong lớp, định nghĩa bên ngoài** với toán tử phạm vi `TenLop::`. Dùng cho hàm dài.

```
void Time::SetTime(int h, int m, int s) { ... }
```

  - Muốn hàm ngoài lớp vẫn inline: thêm từ khóa `inline` trước định nghĩa.

- **Tổ chức mã nguồn** cho dự án nhiều tập tin:

Tập tin	Nội dung
`time.h`	Khai báo lớp (giao diện) – người dùng chỉ cần đọc tập tin này
`time.cpp`	Định nghĩa các hàm thành viên (cài đặt) – `#include "time.h"`
`main.cpp`	Chương trình sử dụng lớp – `#include "time.h"`

```
// time.h
#ifndef TIME_H        // chống include nhiều lần
#define TIME_H
class Time { ... };
#endif
```

- Biên dịch: `g++ main.cpp time.cpp -o ct`.
- **Lợi ích:** tách **giao diện** khỏi **cài đặt**: sửa time.cpp (đổi thuật toán) không phải sửa main.cpp; có thể phân phối time.h + thư viện đã biên dịch mà không lộ mã nguồn.

3.1 Các nhãn public, private và protected

Nhãn	Truy cập được từ	Dùng cho
public	Mọi nơi (hàm thành viên, hàm ngoài lớp, main…)	**Giao diện**: các dịch vụ lớp cung cấp
private	Chỉ hàm thành viên của **chính lớp** đó (và hàm/lớp friend – buổi 6)	**Dữ liệu** và các hàm tiện ích nội bộ
protected	Như private, cộng thêm các **lớp dẫn xuất**	Kế thừa (buổi 9)

- Các nhãn có thể xuất hiện **nhiều lần**, theo thứ tự bất kỳ; mỗi nhãn có hiệu lực đến nhãn kế tiếp.
- **Mặc định:** thành viên của **class** là **private**; thành viên của **struct** là **public**. Đây là **khác biệt duy nhất** giữa class và struct trong C++.
- Điều khiển truy cập áp dụng theo **lớp**, không theo đối tượng: hàm thành viên của PhanSo truy cập được `b.Tu` của **một phân số khác** b.

Ví dụ 4.3 – Lỗi truy cập (g++)

```
int main()
{  Time t;
   t.Hour = 29;
   // error: 'int Time::Hour' is private within this
context
   cout << t.Minute;
   // error: 'int Time::Minute' is private within this
context
   t.SetTime(18, 30, 0);   // hợp lệ: public
}
```

Nguyên tắc che giấu thông tin

Thành viên dữ liệu luôn để **private**; chỉ đưa ra **public** những hàm mà người dùng lớp thật sự cần. Lớp càng ít phơi bày chi tiết, càng dễ thay đổi cài đặt mà không ảnh hưởng chương trình khác.

3.2 Lợi ích của điều khiển truy cập – ví dụ thay đổi cài đặt

Ví dụ 4.4 – Đổi cách biểu diễn Time mà main() không phải sửa

```
// Phiên bản 1: lưu 3 số
class Time
{  public:
      void SetTime(int h, int m, int s);
      int GetHour();
      int GetMinute();
   private:
      int Hour, Minute, Second;
};
int Time::GetHour() { return Hour; }
```

```
// Phiên bản 2: chỉ lưu số giây từ 0 giờ
class Time
{  public:
      void SetTime(int h, int m, int s);
      int GetHour();
      int GetMinute();
   private:
      long Giay;          // 0 .. 86399
};
void Time::SetTime(int h, int m, int s)
{  Giay = h * 3600L + m * 60 + s;  }
int Time::GetHour()   { return Giay / 3600; }
int Time::GetMinute() { return Giay % 3600 / 60; }
```

```
// main.cpp – KHÔNG ĐỔI với cả hai phiên bản
int main()
{  Time t;
   t.SetTime(13, 27, 6);
   cout << t.GetHour() << "h"
        << t.GetMinute();       // 13h27
}
```

- Phiên bản 2 giúp tính **khoảng cách** giữa hai thời điểm rất đơn giản (trừ hai số giây) – có thể đổi sang mà **không** ảnh hưởng người dùng lớp.
- Nếu Hour là public và main viết `t.Hour`, việc đổi sang phiên bản 2 sẽ làm **hỏng** mọi chỗ dùng `t.Hour`.
- Đây chính là **trừu tượng hóa dữ liệu** (buổi 1) được C++ bảo đảm.

4.1 Hàm truy cập (get/set) và hàm tiện ích

- **Hàm truy cập (access functions):**
  - **get** (accessor, hàm truy vấn): trả về giá trị thành viên dữ liệu – `int GetHour()`.
  - **set** (mutator, hàm cập nhật): gán giá trị **có kiểm tra** – `void SetHour(int h)`.
  - Hàm **vị từ** (predicate): kiểm tra điều kiện – `bool IsEmpty()`, `bool LaBuoiSang()`.
- **Vì sao không để dữ liệu public mà dùng get/set?**
  - set **kiểm tra hợp lệ** (giờ 0–23) ⇒ đối tượng không bao giờ sai.
  - Có thể chỉ cho đọc (có get, không có set).
  - Có thể đổi cách lưu trữ bên trong (ví dụ 4.4).
  - Có thể ghi nhật ký, đếm số lần sửa…

- **Hàm tiện ích (utility / helper functions):** hàm **private** hỗ trợ các hàm public khác, người dùng lớp **không** cần và **không** được gọi.

Ví dụ 4.5 – Lớp TaiKhoan

```
class TaiKhoan
{  public:
      long GetSoDu() { return SoDu; }       // get
      int  RutTien(long t);                 // set có kiểm
tra
      bool BiKhoa() { return SoLanSaiPIN >= 3; } // vị từ
   private:
      long SoDu;
      int  SoLanSaiPIN;
      void GhiNhatKy(const char *thaoTac);  // tiện ích
};
```

- `GhiNhatKy` được RutTien, NapTien gọi bên trong; main gọi `tk.GhiNhatKy(…)` ⇒ **lỗi** private.

4.2 Ví dụ hoàn chỉnh: lớp PhanSo

```
class PhanSo
{  private:
      int Tu, Mau;
      int UCLN(int a, int b);     // tiện ích
      void RutGon();              // tiện ích
   public:
      void Set(int t, int m);
      int  GetTu()  { return Tu; }      // get
      int  GetMau() { return Mau; }
      void SetTu(int t) { Tu = t; RutGon(); }
      int  SetMau(int m);               // set có kiểm tra
      PhanSo Cong(PhanSo b);
      double GiaTri() { return (double)Tu / Mau; }
      void In() { cout << Tu << "/" << Mau; }
};
int PhanSo::UCLN(int a, int b)
{  if (a < 0) a = -a;  if (b < 0) b = -b;
   while (b != 0) { int r = a % b; a = b; b = r; }
   return a == 0 ? 1 : a;
}
void PhanSo::RutGon()
{  int u = UCLN(Tu, Mau);
   Tu /= u;  Mau /= u;
   if (Mau < 0) { Tu = -Tu; Mau = -Mau; }
}
```

```
void PhanSo::Set(int t, int m)
{  Tu = t;  Mau = (m != 0) ? m : 1;
   RutGon();
}
int PhanSo::SetMau(int m)
{  if (m == 0) return 0;            // từ chối
   Mau = m;  RutGon();  return 1;
}
PhanSo PhanSo::Cong(PhanSo b)
{  PhanSo c;
   c.Set(Tu * b.Mau + b.Tu * Mau, Mau * b.Mau);
   return c;
}
int main()
{  PhanSo a, b;
   a.Set(6, -8);  b.Set(1, 6);
   a.In(); cout << " + "; b.In(); cout << " = ";
   PhanSo c = a.Cong(b);  c.In();
   cout << " = " << c.GiaTri() << endl;
   cout << a.SetMau(0) << " "; a.In(); cout << endl;
   cout << c.GetTu() << " " << c.GetMau() << endl;
}
```

```
-3/4 + 1/6 = -7/12 = -0.583333
0 -3/4
-7 12
```

- `Set(6, -8)` ⇒ −3/4; `SetMau(0)` bị **từ chối**.

- **Bất biến của lớp** (class invariant): điều kiện **luôn đúng** với mọi đối tượng sau mỗi hàm public:
  - Mau > 0 (dấu nằm ở tử).
  - Tu/Mau **tối giản**.
- Mọi hàm làm thay đổi Tu, Mau (Set, SetTu, SetMau, Cong) đều gọi **RutGon** ⇒ bất biến được giữ.
- Nhờ bất biến: so sánh bằng nhau chỉ cần `Tu == b.Tu && Mau == b.Mau` (−3/4 và 6/−8 có cùng biểu diễn).

Ví dụ 4.6 – Hỏi: nếu Tu, Mau là public?

`a.Mau = 0;` ⇒ `a.GiaTri()` chia cho 0; `a.Tu = 2; a.Mau = 4;` ⇒ phân số không tối giản, so sánh `1/2 == 2/4` cho kết quả **sai**. ⇒ Che giấu dữ liệu không chỉ là "bảo mật" mà là **giữ tính đúng đắn** của đối tượng.

- **Câu hỏi thiết kế khi viết một lớp:**

Câu hỏi	PhanSo
Dữ liệu gì? (private)	Tu, Mau
Bất biến?	Mau > 0, tối giản
Dịch vụ cung cấp? (public)	Set, Get, Cong, Tru, Nhan, Chia, SoSanh, In, GiaTri
Hàm hỗ trợ nội bộ? (private)	UCLN, RutGon
Hàm nào chỉ đọc?	GetTu, GetMau, GiaTri, In – buổi 6: khai báo **const**

- **Hạn chế hiện tại** (sẽ khắc phục):
  - Phải gọi `a.Set(…)` sau khi khai báo, nếu quên ⇒ Tu, Mau rác ⇒ **constructor** (buổi 5).
  - Viết `a.Cong(b)` thay vì `a + b` ⇒ **đa năng hóa toán tử** (buổi 7).

4.4 Lỗi thường gặp khi định nghĩa lớp

Lỗi	Ví dụ	Thông báo / hậu quả
Quên dấu `;` sau `}` của lớp	`class A { … }` rồi `int main()`	Lỗi khó hiểu ở dòng **sau** lớp
Quên `TenLop::` khi định nghĩa ngoài lớp	`void SetTime(int h, …) { Hour = h; }`	'Hour' was not declared in this scope (thành hàm toàn cục)
Truy cập private từ ngoài	`t.Hour = 5;`	'int Time::Hour' is private within this context
Khởi tạo thành viên trong khai báo (C++ cũ)	`class A { int x = 0; };`	Lỗi với C++98; C++11 cho phép
Gọi hàm thành viên không qua đối tượng	`PrintMilitary();` trong main	'PrintMilitary' was not declared
Nhầm `.` và `->`	`p.SetTime(…)` với p là con trỏ	request for member … which is of pointer type
Prototype và định nghĩa khác tham số	khai báo `SetTime(int,int,int)`, định nghĩa `SetTime(int,int)`	no declaration matches

Ví dụ 4.7 – Tìm lỗi

```
class Diem
{  int x, y;
   void Set(int a, int b) { x = a; y = b; }
}
int main() { Diem d; d.Set(1, 2); cout << d.x; }
```

→ **3 lỗi:** thiếu `;` sau `}`; Set và x, y đều **private** (mặc định của class) nên `d.Set(…)`, `d.x` không truy cập được. Sửa: thêm `public:` trước Set, thêm hàm `GetX()`, thêm `;`.

5.1 Tóm tắt buổi 4

- **Lớp** = dữ liệu + hàm thành viên + điều khiển truy cập; **đối tượng** là thể hiện của lớp.
- `public` – giao diện; `private` – cài đặt (mặc định của class); `protected` – cho lớp dẫn xuất.
- Hàm thành viên định nghĩa trong lớp (inline) hoặc ngoài lớp với `TenLop::`; tách .h / .cpp.
- Truy cập thành viên bằng `.` (đối tượng, tham chiếu) và `->` (con trỏ); sizeof đối tượng chỉ gồm dữ liệu.
- **Hàm truy cập** get/set (có kiểm tra), hàm vị từ; **hàm tiện ích** private; giữ **bất biến** của lớp.

struct (C)	class (C++)
Dữ liệu tách hàm	Dữ liệu + hàm trong một đơn vị
Mọi nơi sửa được dữ liệu	Dữ liệu private, sửa qua hàm có kiểm tra
Không tự khởi tạo	Constructor tự khởi tạo
Đổi cấu trúc ⇒ sửa mọi nơi	Đổi cài đặt, giữ giao diện

Câu hỏi nhanh

- Kết quả `sizeof` của một đối tượng PhanSo (2 int, 10 hàm thành viên)? → **8** byte.
- Hàm `UCLN` để private hay public? Vì sao? → private: người dùng phân số không cần; nếu public thì sau này khó thay đổi.

5.2 Bài thực hành buổi 4 – Cài đặt lớp Time và PhanSo

Bài 1 – Lớp Time

- a) Cài đặt lớp Time như slide (constructor, SetTime, PrintMilitary, PrintStandard); tổ chức thành 3 tập tin time.h, time.cpp, main.cpp.
- b) Bổ sung hàm truy cập `GetHour/GetMinute/GetSecond`, `SetHour/SetMinute/SetSecond` có kiểm tra.
- c) Bổ sung `void Tick()` tăng 1 giây (xử lý 23:59:59 → 00:00:00); viết vòng lặp gọi Tick 3700 lần từ 23:00:00 và in kết quả (đáp số **00:01:40**).
- d) Viết lại lớp theo **phiên bản 2** (lưu số giây) mà không sửa main.cpp; chạy lại để kiểm chứng.

Bài 2 – Lớp PhanSo

- a) Cài đặt lớp PhanSo như slide; bổ sung `Tru`, `Nhan`, `Chia` (từ chối chia cho phân số 0), `SoSanh` (trả −1, 0, 1), `Nhap()`.
- b) Viết chương trình nhập mảng n phân số, in tổng, phân số lớn nhất và sắp xếp tăng dần.
- c) Thử gán `a.Tu = 5;` trong main, ghi lại thông báo lỗi.

Gợi ý kiểm thử

Kiểm tra các trường hợp biên: mẫu âm, tử bằng 0 (0/5 ⇒ 0/1), mẫu bằng 0 (bị từ chối), cộng hai phân số đối nhau (kết quả 0/1).

5.3 Câu hỏi và bài tập

Câu hỏi

- 1) So sánh struct và class trong C++. Khác biệt duy nhất về mặt ngôn ngữ là gì?
- 2) Vì sao thành viên dữ liệu nên để private? Nêu ít nhất 3 lý do.
- 3) Phân biệt hàm truy cập và hàm tiện ích. Hàm tiện ích nên đặt ở vùng nào?
- 4) Hàm thành viên định nghĩa trong khai báo lớp có gì đặc biệt?

Bài tập lập trình

- 1) Xây dựng lớp **HinhChuNhat** (dài, rộng > 0): set/get có kiểm tra, ChuVi, DienTich, LaHinhVuong (vị từ), In.
- 2) Xây dựng lớp **SinhVien** (MaSV, HoTen, 3 điểm thành phần): nhập, tính điểm trung bình (hàm tiện ích làm tròn 1 chữ số), xếp loại, in.
- 3) Xây dựng lớp **Date** (ngày, tháng, năm) với hàm tiện ích private `SoNgayTrongThang()`, `LaNamNhuan()`; hàm `NgayKeTiep()`; kiểm tra 28/2/2024 → **29/2/2024**, 31/12/2025 → **1/1/2026**.